Reviewer Pack — Minimal Order of Non-Crossing Partitions and Communication R...

Entry c90ce4806deb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 04:34:27 UTC

Minimal Order of Non-Crossing Partitions and Communication Rank Variance

Entry ID: `c90ce4806deb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 04:34:27 UTC

1. Conjecture

Field A (mathematical branch): Combinatorics (Non-Crossing Partitions) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

> For all k-regular graphs G, the minimal order of non-crossing partitions is linearly related to the communication rank variance, such that |O(G)| = Θ(RankVar(G))

Rationale (proposer's reasoning):

> Non-crossing partitions provide a combinatorial structure for encoding communication protocols. This conjecture suggests that the complexity of these partitions could offer insights into communication complexity measures like rank variance, which quantifies the variation in matrix ranks within a set.

Taxonomy category: `INNOVATIVE_CONSTRUCTION` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `4e94ed6adb6341b6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all k-regular graphs G with n ≤ 40, the correlation coefficient between |O(G)| and RankVar(G) exceeds 0.7 and there is no seed where |O(G)| / RankVar(G) > 1.5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "non-crossing partitions" AND "communication complexity" AND "matrix rank variance" - "k-regular graphs" AND minimal order AND communication rank variance" - "combinatorics" AND communication rank variance AND non-crossing partitions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.0s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations, permutations

def generate_k_regular_graph(n, k):
    if n * k % 2 != 0:
        raise ValueError("Invalid parameters for generating a d-regular graph")

    edges = set()
    while len(edges) < (n * k) // 2:
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))

    adjacency_matrix = [[0] * n for _ in range(n)]
    for u, v in edges:
        adjacency_matrix[u][v] = 1
        adjacency_matrix[v][u] = 1

    return adjacency_matrix

def is_non_crossing_partition(partition):
    # Check if the partition is non-crossing
    for i in range(len(partition)):
        for j in range(i + 1, len(partition)):
            if any(u < v < w or u > v > w for u, v, w in combinations(sorted(partition[i] + partit
                return False
    return True

def min_non_crossing_partition(graph):
    n = len(graph)
    vertices = list(range(n))
```

```
    def backtrack(current_partition, remaining_vertices):
        if not remaining_vertices:
            if is_non_crossing_partition(current_partition):
                return current_partition
            else:
                return None

        min_order = float('inf')
        best_partition = None

        for i in range(1, len(remaining_vertices) + 1):
            for subset in combinations(remaining_vertices, i):
                new_partition = current_partition + [subset]
                result = backtrack(new_partition, list(set(remaining_vertices) - set(subset)))
                if result is not None:
                    order = sum(len(part) for part in result)
                    if order < min_order:
                        min_order = order
                        best_partition = result

        return best_partition

    partition = backtrack([], vertices)
    return partition, len(partition)

def communication_rank_variance(graph):
    n = len(graph)
    rank_variances = []

    for i in range(n):
        row = [graph[i][j] for j in range(n)]
        col = [graph[j][i] for j in range(n)]

        if sum(row) == 0 or sum(col) == 0:
            rank_variances.append(0)
        else:
            row_rank = len(set(row))
            col_rank = len(set(col))
            rank_variances.append((row_rank - 1) * (col_rank - 1))

    return sum(rank_variances)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    metric_name = "Correlation Coefficient"
    instances_tested = 0
    total_metric_value = 0.0
    max_n = 0

    for n in n_values:
        for _ in range(5):  # Ensure at least 30 instances per seed
            graph = generate_k_regular_graph(n, k=2)
            partition, order = min_non_crossing_partition(graph)
            rank_variance = communication_rank_variance(graph)
```

```
            if order == float('inf'):
                continue

            max_n = max(max_n, n)
            instances_tested += 1
            total_metric_value += abs(order - rank_variance) / (order + rank_variance)

    mean_metric_value = total_metric_value / instances_tested
    conjecture_holds = mean_metric_value < 0.7 or any(abs(order - rank_variance) / (order + rank_v
    counterexample = "mapping_undefined" if conjecture_holds else ""

    return {
        "metric_name": metric_name,
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if not r["conjecture_holds"]) / len(results)

    if all(not r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means it was unable to complete the required computations to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14060
2	preregistration	ollama_remote	glm4:latest	0	0	14341
3	novelty	ollama_remote	glm4:latest	0	0	8749
4	novelty	ollama_remote	glm4:latest	0	0	11905
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20157
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13277
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18056
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14544
9	judge	ollama_remote	glm4:latest	0	0	16381

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 131469 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c90ce4806deb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c90ce4806deb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c90ce4806deb.tar.gz` (if generated)